

SymPy Bug Report

1 Bug 21: solveset returns 1 for unsatisfiable principal-log equation

1.1 Minimal Reproducer

```
import sympy
from sympy import Eq, I, S, log, pi, simplify, solveset, symbols

x = symbols("x")
eq = Eq(log(x), 2*pi*I)
sol = solveset(eq, x, domain=S.Complexes)

print("SymPy version:", sympy.__version__)
print("solveset result:", sol)
print("residual at returned value 1:",
      simplify(eq.lhs.subs(x, 1) - eq.rhs.subs(x, 1)))
```

1.2 Observed Behavior

```
SymPy version: 1.14.0
solveset result: {1}
residual at returned value 1: -2*I*pi
```

SymPy returns the finite solution set $\{1\}$. However, direct substitution of the returned value into the original equation gives

$$\log(1) - 2\pi i = -2\pi i \neq 0.$$

Thus the returned element does not satisfy the equation that `solveset` was asked to solve.

1.3 Expected Behavior

The mathematically correct result is $\emptyset$, represented in SymPy as `EmptySet`. There is no complex value x for which SymPy's principal logarithm satisfies $\log(x) = 2\pi i$.

1.4 Mathematical Explanation

SymPy's complex log is the principal logarithm. For $x \neq 0$, it can be written as

$$\log(x) = \ln|x| + i \operatorname{Arg}(x), \quad \operatorname{Arg}(x) \in (-\pi, \pi].$$

Therefore every value in the image of the principal logarithm has imaginary part in $(-\pi, \pi]$. The target $2\pi i$ has imaginary part 2π , so it lies outside that image.

Exponentiating both sides gives $x = e^{2\pi i} = 1$, but this implication is not reversible for the principal logarithm. The value $x = 1$ satisfies $e^{\log(1)} = 1$, but it does not satisfy the original equation because $\log(1) = 0$, not $2\pi i$. The wrong answer is therefore not a harmless branch representation or an equivalent form of a solution; it is a concrete non-solution.

1.5 Source-Level Diagnosis

The diagnosis identifies the relevant code path in `sympy/solvers/solveset.py`, in the complex inversion helper `_invert_complex` near line 550. The traced call path is that `solveset` rewrites `Eq(log(x), 2*pi*I)` as `log(x) - 2*pi*I`, calls `_solveset(..., _check=True)`, and then reaches the generic inversion branch. That branch calls `_invert` over `S.Complexes`, which dispatches to `_invert_complex`.

For this equation, `_invert_complex(log(x) - 2*pi*I, {0}, x)` first removes the additive constant and then handles `log(x)` through the generic function-inverse path. It returns the transformed solution $(x, \{e^{2\pi i}\})$, which simplifies to $(x, \{1\})$. The problematic source rule is the generic use of a function's inverse when one is available:

```
if hasattr(f, 'inverse') and f.inverse() is not None and \
    not isinstance(f, TrigonometricFunction) and \
    not isinstance(f, HyperbolicFunction) and \
    not isinstance(f, exp):
    ...
    return _invert_complex(f.args[0],
                           imageset(Lambda(n, f.inverse()(n)), g_ys), symbol)
```

This treats `exp` as a globally valid inverse for `log` over the complex numbers. The missing condition is that the right-hand side must lie in the principal branch range of `log`. The later `_check=True` filtering identified in the diagnosis only performs a domain or singularity check through `domain_check`; it does not perform a residual check against the original equation for the fully inverted finite set. Consequently, the candidate 1 is accepted even though substituting it into the original equation gives a nonzero residual. A plausible fix direction is to make the complex inverse path for `log` branch-range aware, or to return a conditional set when the branch condition cannot be resolved. The diagnosis also supports a defensive residual check for finite candidates produced by branch-sensitive inversion.

1.6 Additional Failing Instances

The same failure occurs for nonzero integer multiples of $2\pi i$. For every nonzero integer n in the verified family, `solveset` is asked to solve

$$\log(x) = 2\pi i n$$

over the complex numbers. SymPy returns $\{1\}$, because $e^{2\pi i n} = 1$, but $2\pi n$ is outside the principal argument interval $(-\pi, \pi]$. Hence the expected solution set is $\emptyset$ in each case.

Input / Expression	SymPy behavior	Expected behavior
<code>solveset(log(x) = 2πi, x, ℂ)</code>	returns {1}; residual $-2\pi i$	$\emptyset$
<code>solveset(log(x) = 4πi, x, ℂ)</code>	returns {1}; residual $-4\pi i$	$\emptyset$
<code>solveset(log(x) = -2πi, x, ℂ)</code>	returns {1}; residual $2\pi i$	$\emptyset$
<code>solveset(log(x) = 6πi, x, ℂ)</code>	returns {1}; residual $-6\pi i$	$\emptyset$
<code>solveset(log(x) = 8πi, x, ℂ)</code>	returns {1}; residual $-8\pi i$	$\emptyset$
<code>solveset(log(x) = 10πi, x, ℂ)</code>	returns {1}; residual $-10\pi i$	$\emptyset$

1.7 Related Issues Assessment

The bug-finding harness performed a best-effort deduplication check. The closest related public material found was documentation confirming that SymPy’s log is principal-valued, documentation describing the contract of `solveset`, and documentation for complex inversion in `solveset`. No public issue, pull request, discussion, or release note was identified that reports this exact reproducer or the exact wrong output {1}. The deduplication verdict is therefore a new instance of a known failure mode: the general need for principal-branch-aware inverse solving is known, but this specific reproducible case is previously unreported and remains individually actionable as a regression test and issue.

1.8 Proposed SymPy Regression Test

```
import pytest

from sympy import Eq, FiniteSet, I, S, log, pi, simplify, solveset, symbols

@pytest.mark.parametrize("n", [1, 2, -1, 3, 4, 5])
def test_solveset_principal_log_rejects_out_of_range_targets(n):
    x = symbols("x")
    eq = Eq(log(x), 2*pi*I*n)
    sol = solveset(eq, x, domain=S.Complexes)

    assert sol != FiniteSet(1)
    assert simplify(eq.lhs.subs(x, 1) - eq.rhs.subs(x, 1)) != 0
```